

COMS20018, Introduction to AI:

Linear Regression and Classification: Part I

Mengyan Zhang

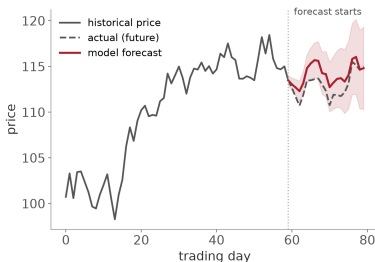
School of Computer Science
University of Bristol

22nd September 2026

Agenda

- ▶ Categories of machine learning
 - ▶ Supervised, unsupervised, reinforcement learning
 - ▶ Self-supervised & semi-supervised learning
 - ▶ Where today fits: regression
- ▶ Model selection
 - ▶ Training, validation & test data
 - ▶ Cross-validation
- ▶ Linear regression: least squares
 - ▶ Curve fitting
 - ▶ Linear basis function models
 - ▶ Least squares
 - ▶ The normal equations
 - ▶ Worked example
 - ▶ Evaluation metrics

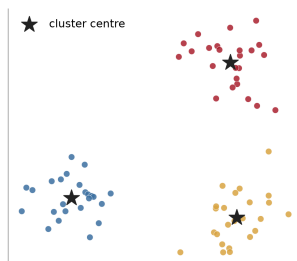
Categories of machine learning (1/3)



Supervised Learning, forecasting a price (regression)

- Given labelled examples $\{(x_n, t_n)\}_{n=1}^N$, learn a mapping $x \mapsto t$.

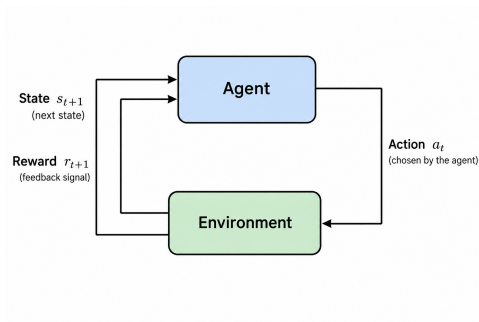
Categories of machine learning (2/3)



Unsupervised Learning, customer segmentation (clustering)

- ▶ Given only *unlabelled* data $\{x_n\}_{n=1}^N$, find structure, e.g. clusters, or the underlying density.

Categories of machine learning (3/3)

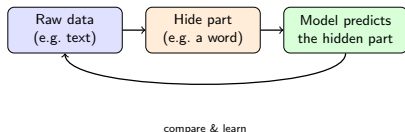


Reinforcement Learning – e.g. AlphaGo (game-playing)

- An agent learns from *reward* signals produced by its own actions, not from labelled examples; covered by Week 9.

Two more flavours: self-supervised & semi-supervised

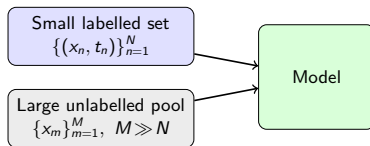
Self-supervised



- ▶ Labels generated automatically from the data itself, e.g. predict the next word, or a masked-out image patch
- ▶ Powers modern LLM pretraining; no human-annotated labels needed

⇒ Both sit between supervised and unsupervised learning, useful whenever labels are scarce but data is not

Semi-supervised

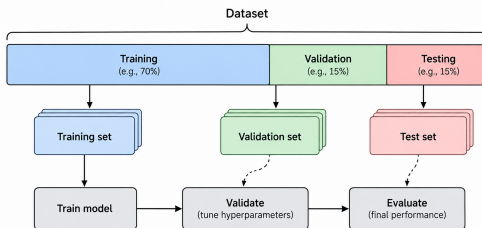


- ▶ Labels are expensive, raw data is cheap; use the unlabelled data to learn structure, then fit it to the few labels you have

Model selection: training, validation & test data

- ▶ Choosing between candidate models or hyperparameters (e.g. the polynomial order M we meet next) is a problem for *any* learning method, not just parametric ones like ours today.

⇒ Split the data into three disjoint sets:

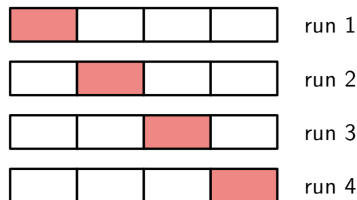


- ▶ **Training set:** fit the parameters $\mathbf{w}$ of each candidate model.
- ▶ **Validation set:** held out from training, used to compare candidates and pick the best one.
- ▶ **Test set:** held out until the very end, used *once* for an unbiased estimate of performance on new data.

Model selection: avoiding biased estimates

- ⇒ Never choose a model using test-set performance; that would bias the estimate of how well it generalises.
- ⇒ Never *test* on the training set either; training error is optimistically biased and does not reflect performance on new data.

Cross-validation



Bishop, *PRML* (2006), Fig. 1.18: 4-fold cross-validation, in each of the 4 runs, one block (red) is held out for validation and the rest (white) used for training.

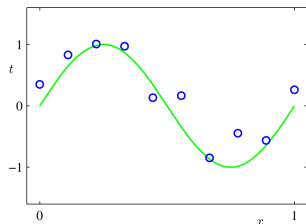
- ▶ Problem: a fixed validation split wastes data that could otherwise be used for training, costly when labelled data is scarce.
- ▶ **S -fold cross-validation:** partition the data into S groups. In turn, train on $S-1$ groups and validate on the remaining one; average performance over the S runs.
- ▶ Extreme case $S=N$: **leave-one-out** cross-validation.

⇒ Cost: training must be repeated S times; fine for cheap models, expensive for large ones (e.g. deep networks).

Today: supervised learning, regression

- ▶ Within supervised learning, the target t can be:
 - ▶ continuous $\Rightarrow$ **regression** (e.g. house price from features, the price-forecasting example above).
 - ▶ discrete (a class label) $\Rightarrow$ **classification** (e.g. spam or not, dog or bagel); next lecture.
- $\Rightarrow$ **Today**: regression, curve fitting and least squares. Classification, and more on regression, follow next lecture.

Motivating example: polynomial curve fitting



Bishop, *Pattern Recognition and Machine Learning* (2006), Fig. 1.2:
 $N=10$ points (blue) sampled from $\sin(2\pi x)$ (green) plus random noise.

- ▶ Training set: N observations of an input x , together with a target t : $\{(x_n, t_n)\}_{n=1}^N$.
- ▶ For our running example, take x and t to be **scalars** (1D), as in the plot on the left, where t_n is generated from $\sin(2\pi x_n)$ plus random noise.
- ▶ Goal: predict t for a *new* x , i.e. learn a function $y(x, \mathbf{w})$.

- ▶ Fit a polynomial of order M :

$$y(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \cdots + w_Mx^M = \sum_{j=0}^M w_jx^j$$

- ▶ $\mathbf{w} = (w_0, w_1, \dots, w_M)^\top$ is the **weight vector**, the $M+1$ parameters we need to learn from the data.

Linear basis function models

- In general:

$$y(x, \mathbf{w}) = \sum_{j=0}^{M-1} w_j \phi_j(x) = \mathbf{w}^\top \boldsymbol{\phi}(x)$$

where $\phi_0(x) = 1$ (a bias term) and $\phi_1, \dots, \phi_{M-1}$ are fixed **basis functions**.

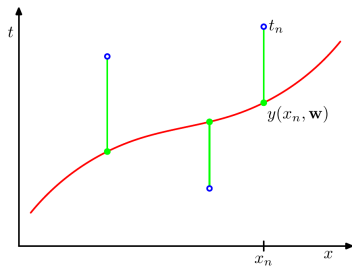
- Common choices: polynomials ($\phi_j(x) = x^j$), Gaussians $\phi_j(x) = \exp\{-(x - \mu_j)^2/2s^2\}$, sigmoidal $\phi_j(x) = \sigma((x - \mu_j)/s)$.
- ⇒ **Linear model** means linear in $\mathbf{w}$, *not* linear in x

Least squares

- ▶ Measure how well $y(x, \mathbf{w})$ fits the data with the **sum-of-squares loss function**:

$$L_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - y(x_n, \mathbf{w})\}^2$$

- ▶ **Least squares**: choose $\mathbf{w}$ to minimise $L_D(\mathbf{w})$.

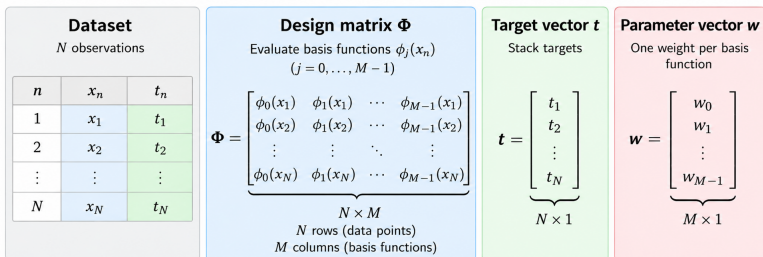


- ▶ Why squared error? Simple to optimise (differentiable, convex in $\mathbf{w}$), and, as we'll see shortly, it falls out naturally once we assume **Gaussian noise**.
- ⇒ Because y is linear in $\mathbf{w}$, L_D is a *quadratic* function of $\mathbf{w}$; it has a unique minimum, solvable in closed form.

Bishop, *PRML* (2006), Fig. 1.3: each green bar is one displacement $\{t_n - y(x_n, \mathbf{w})\}$, squared and summed to give $L_D(\mathbf{w})$.

Solving least squares: the design matrix

- Stack the data: design matrix $\Phi \in \mathbb{R}^{N \times M}$ with $\Phi_{nj} = \phi_j(x_n)$, target vector $\mathbf{t} = (t_1, \dots, t_N)^\top$. Then $L_D(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2$.



Solving least squares: the normal equations

- ▶ Recall $L_D(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2$. Differentiating w.r.t. $\mathbf{w}$:

$$\nabla_{\mathbf{w}} L_D(\mathbf{w}) = \Phi^\top \Phi \mathbf{w} - \Phi^\top \mathbf{t}$$

- ▶ Setting $\nabla_{\mathbf{w}} L_D(\mathbf{w}) = 0$ gives the **normal equations**:

$$\Phi^\top \Phi \mathbf{w} = \Phi^\top \mathbf{t}$$

- ▶ Solution (the **least-squares**, or maximum-likelihood, estimate):

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \Phi^\dagger \mathbf{t}$$

where $\Phi^\dagger$ is the (Moore-Penrose) **pseudo-inverse** of Φ .

See Bishop, *PRML* (2006), Section 3.1.1, for the full derivation.

Try it: least squares by hand

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

- ▶ Before we go through it together: write down $\mathbf{t}$ and Φ for this data, then use $\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t}$ to find w_0 and w_1 .
- ⇒ Try it on paper – we'll work through the steps on the next slide.

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^T \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

Least squares: a worked example

Example

Fit a straight line $y(x, \mathbf{w}) = w_0 + w_1 x$ (i.e. basis functions $\phi_0(x)=1$, $\phi_1(x)=x$, so $M=2$) to the data:

x_n	-1	0	1	2
t_n	0	1	3	9

$$\mathbf{t} = \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} \quad \Phi = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} \quad \Phi^\top \Phi = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 6 \end{bmatrix}$$

$$\mathbf{w}_{\text{ML}} = (\Phi^\top \Phi)^{-1} \Phi^\top \mathbf{t} = \frac{1}{20} \begin{bmatrix} 6 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 3 \\ 9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.9 \end{bmatrix}$$

$$\Rightarrow y(x, \mathbf{w}_{\text{ML}}) = 1.8 + 2.9x$$

Example adapted from the *Machine Learning* unit (COMS30035).

How to evaluate a regression task: evaluation metrics

- ▶ **Mean squared error (MSE):**

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N \{t_n - y(x_n, \mathbf{w})\}^2$$

- ▶ Same shape as $L_D(\mathbf{w})$ (up to the $\frac{1}{2}$ and the averaging); squaring means large errors are penalised heavily.
- ▶ **Root mean squared error (RMSE):** $\sqrt{\text{MSE}}$, back in the same units as t (this is E_{RMS} from Fig. 1.5).
- ▶ **Mean absolute error (MAE):** More robust to outliers than MSE/RMSE, since errors are not squared.

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^N |t_n - y(x_n, \mathbf{w})|$$

- ▶ **R^2 (coefficient of determination):** the fraction of the variance in t explained by the model; $R^2=1$ is a perfect fit, $R^2=0$ is no better than always predicting the mean of t .

Summary

- ▶ **Categories of ML:** supervised, unsupervised, reinforcement, self-/semi-supervised
 - ▶ **Model selection:** train / validation / test split; cross-validation; never touch the test set until the end
 - ▶ **Regression:** linear-in- $\mathbf{w}$ models, $y(x, \mathbf{w}) = \mathbf{w}^\top \phi(x)$
 - ▶ **Least squares:** minimise $L_D(\mathbf{w}) = \frac{1}{2} \|\mathbf{t} - \Phi \mathbf{w}\|^2 \Rightarrow \mathbf{w}_{\text{ML}} = \Phi^\dagger \mathbf{t}$
 - ▶ **Evaluation:** MSE/RMSE/MAE and R^2 , always on held-out data
- ⇒ Next lecture: the probabilistic view of least squares, stochastic gradient descent, regularization, and classification (logistic regression).

Reading

- ▶ Bishop, C. M., *Pattern Recognition and Machine Learning* (2006).
[Free online.](#)
- ▶ Today's material: Chapter 1 (§1.1, §1.3), Chapter 3 (§3.1).